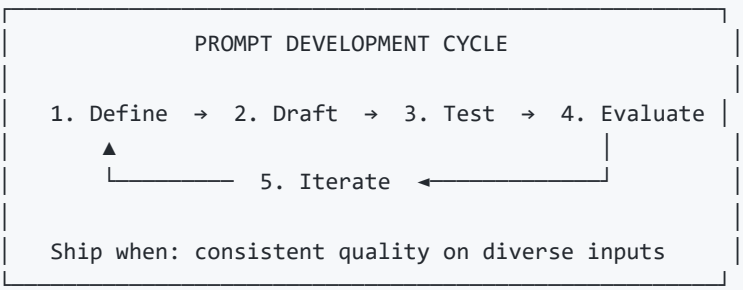
Interview Cheat Sheet — Prompt & Agent Development

> 📋 **Purpose**: Quick-reference bullets for interview discussions on prompt engineering and agent development lifecycles.

Prompt Development Cycle



1. Define the Task

- Clarify **input/output contract** — what goes in, what comes out
- Identify **edge cases** upfront (empty input, adversarial input, multilingual)
- Set **success criteria** — accuracy threshold, latency budget, cost limit
- Decide **determinism needs** — creative (high temp) vs factual (low temp)

2. Draft the Prompt

- Start with a **clear role** ("You are a senior code reviewer...")
- Add **explicit output format** (JSON schema, markdown, bullet list)
- Include **constraints** ("Do NOT include explanations outside the JSON")
- Add **2-3 few-shot examples** if zero-shot is inconsistent
- Put critical instructions at the **start AND end** (lost-in-the-middle effect)

3. Test Systematically

- Build a **test suite** of 20-50 diverse inputs (happy path + edge cases)
- Test with **adversarial inputs** (prompt injection, unexpected formats)
- Test at **different temperatures** to find the sweet spot

- Check **token count** — will it fit within context window + response?

4. Evaluate

- **Automated metrics**: exact match, regex validation, JSON parse success
- **LLM-as-judge**: use a stronger model to grade outputs (cheaper than humans)
- **Human evaluation**: for subjective quality (tone, helpfulness, safety)
- Track **failure modes**: hallucination rate, format breaks, refusals

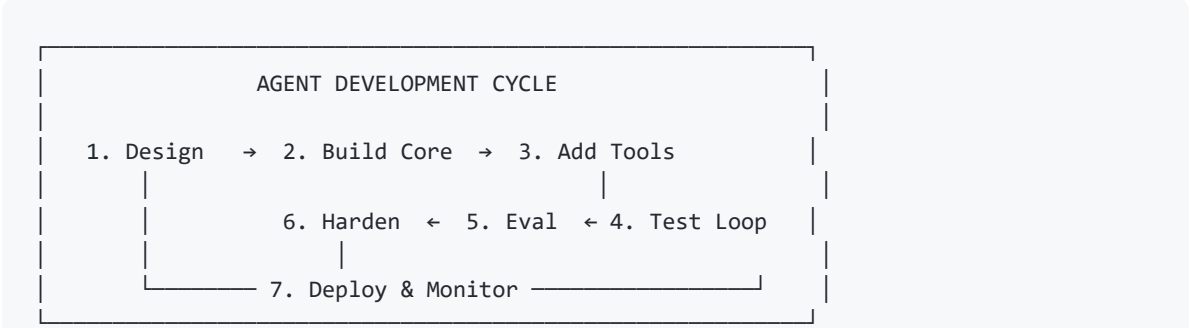
5. Iterate

- Analyze failures → adjust prompt (add examples, clarify constraints)
- If 10+ examples don't help → consider **fine-tuning**
- If hallucination persists → add **RAG** (ground in real data)
- Version control your prompts — they are code

Key Interview Points

- "I treat prompts as **software artifacts** — versioned, tested, evaluated"
- "I start zero-shot, add few-shot if needed, consider fine-tuning last"
- "I always build an **eval suite** before optimizing — otherwise I'm guessing"
- "Critical instructions go at the start AND end to avoid the lost-in-the-middle problem"

Agent Development Cycle



1. Design the Agent

- Define the **task scope** — what the agent should and SHOULD NOT do
- Choose **architecture pattern**:
  - Simple task → **ReAct** (think → act → observe loop)
  - Complex multi-step → **Planner/Executor** (plan first, then execute)

    - Multiple skills → **Multi-agent** (specialist agents + orchestrator)
- Define **tools** the agent needs (APIs, databases, calculators, code exec)
- Set **guardrails** — budget limits, action restrictions, timeout

2. Build Core Loop

- Implement the **system prompt** (role, available tools, output format)
- Build the **tool interface** (name, description, parameters, execution)
- Implement the **agent loop** (LLM call → parse action → execute → observe)
- Add **stopping conditions** (max iterations, "FINAL ANSWER" token, timeout)

3. Add Tools

- Each tool needs: **clear name**, **precise description**, **typed parameters**
- Tool descriptions are prompts — invest in making them unambiguous
- Return **structured results** (not raw HTML, not 10K-line dumps)
- Add **error messages** that help the agent self-correct

4. Test the Loop

- Test with **simple tasks first** (single tool, 1-2 steps)
- Gradually increase complexity (multi-tool, 5+ steps)
- Test **error recovery** — what happens when a tool fails?
- Test **edge cases** — ambiguous queries, impossible tasks, conflicting info
- Log every step: thought → action → observation (essential for debugging)

5. Evaluate

- **Task completion rate** — does it actually solve the problem?
- **Step efficiency** — how many steps vs optimal?
- **Tool selection accuracy** — does it pick the right tool?
- **Cost per task** — total tokens used × price
- **Failure analysis** — categorize WHY failures happen (loops, wrong tool, hallucinated action)

6. Harden

- Add **input guardrails** (prompt injection detection, input validation)
- Add **output guardrails** (content filtering, action allowlists)
- Add **cost controls** (max tokens per run, max API calls)
- Add **human-in-the-loop** for high-stakes actions (delete, purchase, send)
- Implement **retry with backoff** for transient failures

- Add **graceful degradation** — if agent fails, return partial results + explanation

7. Deploy & Monitor

- **Log everything** — inputs, thoughts, actions, observations, outputs
- Monitor **latency, cost, success rate, error rate** in production
- Set up **alerts** for unusual patterns (cost spikes, loops, new error types)
- Build a **feedback loop** — user corrections improve future prompts
- **A/B test** prompt changes before full rollout

Key Interview Points

- "I start with ReAct for simple tasks, escalate to Planner/Executor for complex ones"
- "Tool descriptions are the most underrated part — they're prompts for the agent"
- "I always build guardrails BEFORE deploying — budget limits, action restrictions, timeouts"
- "I treat agent failures as a **classification problem** — categorize, then fix systematically"
- "Observability is non-negotiable — if I can't see every thought/action/observation, I can't debug"

Quick Comparison Table

	Prompt Engineering	Agent Development
Core artifact	The prompt text	System prompt + tools + loop + guardrails
Success metric	Output quality	Task completion rate
Failure mode	Hallucination, format break	Loops, wrong tool, goal drift
Iteration unit	Prompt version	Tool/prompt/loop
Testing approach	Input/output pairs	End-to-end scenarios
When to use	Single-turn tasks	Multi-step, dynamic tasks needing tools
Cost driver	Token count × calls	Steps × tokens/step
Key skill	Clarity of instructions	Error handling & recovery design

---

One-Liner Answers for Common Interview Questions

"How do you approach building a new prompt?" → Define success criteria, start zero-shot, test on 20+ examples, add few-shot if needed, iterate on failures.

"How do you evaluate prompt quality?" → Automated eval suite (exact match, LLM-as-judge) + human review for subjective quality. Always measure before and after changes.

"What's the difference between an agent and a chatbot?" → A chatbot responds to messages. An agent can decide what to do, use tools, observe results, and loop until the task is done — autonomously.

"How do you handle agent failures?" → Max iteration limits, tool-level error handling with helpful messages, graceful degradation, and human-in-the-loop for high-stakes actions.

"When would you use multi-agent vs single-agent?" → Single agent for tasks one person could do in one sitting. Multi-agent when you need distinct expertise, peer review, or parallel execution.

"How do you prevent prompt injection?" → Input sanitization, separate system/user messages, instruction hierarchy (system > user), output validation, and never execute LLM output as code without sandboxing.

"How do you manage cost?" → Use the cheapest model that meets quality bar, cache repeated queries, set per-request token budgets, route simple tasks to smaller models (cost-based routing).

← Back to AI & Agents Syllabus